Algoritmo Genético Simple

Ejemplo resuelto

Maximizar la función

$$f(x,y) = 21.5 + x * sen(4 * PI * x) + y * sen(20 * PI * y)$$

Características a considerar:

1. Optimizar una función (maximizar);
2. Número de variables independientes (x,y);
3. La precisión en decimales a usar para cada una de las variables es de cuatro;
4. Los intervalos de cada una de las variables. X=[-3.0,12.1] y Y=[4.1,5.8]

Para este problema considerar las siguientes características del AGS:

1. Utilizar una representación binaria,
2. El método de selección será un método de selección proporcional,
3. La cruza desde un punto,
4. La mutación es aleatoria y
5. Parámetros del algoritmo genético (AG).
    a. número de generaciones (Generaciones),
    b. tamaño de la población o número de individuos,
    c. la probabilidad de cruza (P_c),
    d. la probabilidad de mutación (P_m).

Algoritmo para resolver el problema:

1. *Generación poblacional inicial*. Se parte de la población inicial,
2. *Iniciar t=0.* en la iteración t=0 o en la generación cero. Esta población inicial, dado que está codificada en binario, se tiene que hacer un mapeo al dominio del problema y,
3. *Evaluar la población inicial*. Posteriormente, evaluar la función.
4. *Hacer ciclo de evolución.* Después entraremos al ciclo evolutivo, en el cual tenemos las siguientes etapas:
    1. Selección,
    2. Cruza,
    3. Mutación y,
    4. *Evaluar la nueva población*. después de mutación, vamos a tener una nueva población, la cual tenemos que decodificar y evaluar;
    5. *Incrementar t, t=t+1*. aquí estaremos incrementando nuestro contador de generaciones.
5. *Repetir mientras (t < Generaciones).* Este proceso se va a repetir un número determinado de generaciones, que se definió al inicio del problema.

Paso a paso de un algoritmo genético básico

1. **Definir los valores de los parámetros**

Definición de Parámetros		Descripción
Nind =100	% Número de individuos en la población	tamaño de la población, un número de individuos
Lind = 20	%Longitud de los individuos	longitud en los individuos, el cual está en función del número de variables y la precisión en cuanto a número de bits ocupados para representar cada variable.
Pc = 0.9	%Probabilidad de cruzamiento	Se deben definir también las probabilidades de cruza Pc. Haciendo una analogía con la parte natural, y recordando lo visto en el módulo 2, el operador de cruza se presenta con mayor frecuencia, por lo que definir una probabilidad de cruza en el rango del 60 al 100% es adecuada.
Pm = 0.01	%Probabilidad de mutación	Se deben definir también las probabilidades de mutación, Pm. En el caso de mutación, las mutaciones en la naturaleza se manifiestan con una menor frecuencia. De esta manera, el rango de mutación se define como un valor muy pequeño tendiendo a cero, hasta el 10% entre la longitud del individuo.
Maxgen = 1000	%Máximo número de generaciones	Esto nos dice que, con la probabilidad de mutación definida como el límite máximo, se estará mutando un bit cada 10 individuos en la población.
Nvar = 2	%Número de variables	Restaría definir el rango de las variables de decisión del caso de estudio, y esto está dado en el problema mismo
Rango = [-3,4,1;12,1,5,8]	%Rango de las variables de un problema	

```
# Valores de las variables
Nind = 100
Lind = 33
Pc = 0.9
Pm = 0.01
Maxgen = 1000
Nvar = 2
rango = np.array([[-3, 4.1], [12.1, 5.8]])
```

2. **Crear la población inicial**

Se crean los nodos(individuos) de la población de forma aleatoria, que son el *espacio de soluciones* potenciales para resolver la función objetivo. Cada nodo debe decodificarse a binario.

```
genotipo = creapob(Nind, Lind)
fenotipo = decodifica (genotipo, rango);
generaciones = 1
```

3. **Obteniendo el valor óptimo**

El valor óptimo corresponde al "mejor valor" encontrado en la función objetivo, así como las variables que permitieron encontrarlo.

```
Mejor = np.full((Maxgen, 1), np.nan) #Referencia al mejor valor óptimo encontrado de la función objetivo
Mejor_cromosoma = np.zeros((Maxgen, Lind)) # Mejor valor (nodo) encontrado por cada generación
objv = objfun(fenotipo) #evaluación de la función
```

4. Ciclo Evolutivo

Una vez creada y evaluada la población inicial, se procede al ciclo evolutivo

```
generaciones = 1

while generaciones < Maxgen: #Numero de generaciones predefinidas
```

Para iniciar cada generación(ciclo), se transforma el valor objetivo a un valor de aptitud. Este mapeo se realiza para garantizar que la mejor solución posea el valor de aptitud más alto

```
aptitud = rankeo(objv, 1) #cada generación se establece el fitness para cada solución encontrada
```

Una vez calculado el valor de aptitud, siendo todos valores positivos, se procede a **seleccionar** a los más aptos al aplicar el método de selección de la ruleta, método de selección proporcional.

```
nuevo_gen = ruleta(genotipo, fenotipo, aptitud)
```

Con el conjunto de individuos seleccionados, se procede a aplicar el operador de **cruza** con probabilidad definida con anterioridad. Al generar un valor aleatorio, si éste es menor o igual a la probabilidad de cruza, el primer par de individuos del conjunto de individuos generado por el método de selección, se cruzarán. En caso contrario, pasarán sin cambios.

```
nuevo_gen = xunpunto(nuevo_gen, Pc)
```

Posterior a la cruza, se procede con el operador de **mutación**. Dada una probabilidad de mutación, si el número aleatorio generado es menor o igual a dicha probabilidad, se mutará. En el caso binario, se cambiará el valor de 1 a 0, o viceversa.

```
nuevo_gen = muta(nuevo_gen, Pm)
```

Hasta este punto ya se tiene una nueva población. Y esta nueva población de volverá a decodificar a binario y se volverá a evaluar.

```
nuevo_feno = decodifica(nuevo_gen, rango)
nuevo_objv = objfun(nuevo_feno)
```

La nueva población creada sustituirá a la población anterior

```
genotipo = nuevo_gen
objv = nuevo_objv
```

Agregar el código necesario para almacenar las mejores soluciones de cada generación e imprimir el mejor valor de todas las generaciones.

```
idx = np.argmax(objv)
Mejor[generaciones] = objv[idx]
Mejor_cromosoma[generaciones, :] = genotipo[idx, :]
generaciones += 1
```

A continuación, se desarrollan las funciones del algoritmo genético:

creapob

```
def creapob(Nind, Lind):
    return np.random.randint(0, 2, size=(Nind, Lind))
```

decodifica

```
def decodifica(genotipo, rango):
    Nvar = rango.shape[1]
    Nind, Lind = genotipo.shape
    Lvar = Lind // Nvar
    potencias = 2 ** np.arange(Lvar)

    fenotipo = np.zeros((Nind, Nvar))

    for i in range(Nind):
        for j in range(Nvar):
            fenotipo[i, j] = np.sum(potencias * genotipo[i, (j * Lvar):(j * Lvar + Lvar)])

    for i in range(Nvar):
        fenotipo[:, i] = rango[i, 0] + ((rango[i, 1] - rango[i, 0]) / (2 ** Lvar - 1)) * fenotipo[:, i]

    return fenotipo
```

La codificación de las variables responde a las siguientes formulas

X	Y
Codificación de las variables $Tamaño = [\log_2((l_{sup} - l_{inf}) \times 10^{precisión})]$ donde: $l_{sup} = 12.1$, $l_{inf} = -3.0$, $precisión = 4$ $Tamaño = [\log_2((12.1 - (-3.0)) \times 10^4)] = 17.2041$	Codificación de las variables $Tamaño = [\log_2((l_{sup} - l_{inf}) \times 10^{precisión})]$ donde: $l_{sup} = 5.8$, $l_{inf} = 4.1$, $precisión = 4$
$x_i = l_{inf} + decimal(11001...11_2)\left[\frac{l_{sup} - l_{inf}}{2^{Lind} - 1}\right]$ donde: $l_{sup} = 12.1$, $l_{inf} = -3.0$, $Lind = 18$	$y_i = l_{inf} + decimal(11001...11_2)\left[\frac{l_{sup} - l_{inf}}{2^{Lind} - 1}\right]$ donde: $l_{sup} = 5.8$, $l_{inf} = 4.1$, $Lind = 15$
$x_i = -3.0 + (144254)\left[\frac{12.1 - (-3.0)}{2^{18} - 1}\right] = 5.3093$	$y_i = 4.1 + (6599)\left[\frac{5.8 - 4.1}{2^{15} - 1}\right] = 4.4423$

Esto significa de la cadena binaria a las variables de decisión en el dominio del problema, se procede a evaluar la función para obtener su valor objetivo.

$$f_1(x_1,y_1) = 21.5 + x_1\, sen(4\pi x_1) + y_1 sen(20\pi y_1)$$

$$f_1(x_1,y_1) = 19.9660 = a_1$$

objfun

```
def objfun(fenotipo):
    Nind, Nvar = fenotipo.shape
    objv = np.zeros((Nind, 1))

    for i in range(Nind):
        objv[i, 0] = 21.5 + fenotipo[i, 0] * np.sin(4 * np.pi * fenotipo[i, 0]) + fenotipo[i, 1] * np.sin(20 * np.pi *
fenotipo[i, 1])

    return objv
```

rankeo

```
def rankeo(objv, direccion):
    SP = 2
    Nind, Nobj = objv.shape
    aptitud = np.zeros((Nind, 1))

    if direccion == 1:
        nuevo_objv = np.sort(objv, axis=0)
    else:
        nuevo_objv = np.sort(-1 * objv, axis=0)

    apt = 2 - SP + 2 * (SP - 1) * (np.arange(Nind) / (Nind - 1))
    posori = np.argsort(nuevo_objv, axis=0)

    aptitud[posori, 0] = apt.reshape(-1, 1)

    return aptitud
```

ruleta

```
def ruleta(genotipo, fenotipo, aptitud):
    Nind, aux = aptitud.shape
    total = np.sum(aptitud)
    probabilidad = aptitud / total
    acumulada = np.cumsum(probabilidad)
    idx = np.zeros((Nind, 1), dtype=int)

    for i in range(Nind):
        selecciona = np.random.rand()
        aux = np.where(acumulada >= selecciona)[0]
        idx[i, 0] = aux[0]

    nuevo_gen = genotipo[idx.flatten(), :]

    return nuevo_gen
```

xunpunto

```
def xunpunto(nuevo_gen, Pc):
    Nind, Lind = nuevo_gen.shape
    aux_gen = np.zeros((Nind, Lind))
    par = Nind % 2

    for i in range(0, Nind - 1, 2):
        cruza = np.random.rand()
        if cruza <= Pc:
```

```
            corte = np.random.randint(0, Lind - 1)
            aux_gen[i, :] = np.concatenate((nuevo_gen[i, :corte], nuevo_gen[i + 1, corte:]))
            aux_gen[i + 1, :] = np.concatenate((nuevo_gen[i + 1, :corte], nuevo_gen[i, corte:]))
        else:
            aux_gen[i, :] = nuevo_gen[i, :]
            aux_gen[i + 1, :] = nuevo_gen[i + 1, :]

    if par == 1:
        aux_gen[Nind - 1, :] = nuevo_gen[Nind - 1, :]

    nuevo_gen = aux_gen

    return nuevo_gen
```

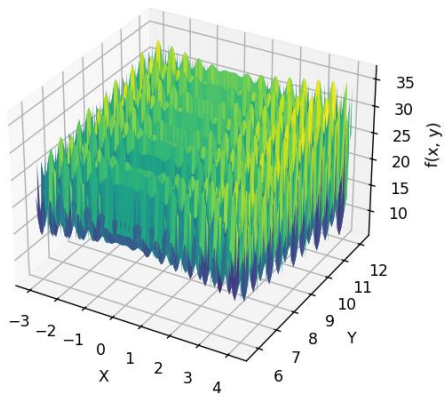
muta

```
def muta(nuevo_gen, Pm):
    Nind, Lind = nuevo_gen.shape
    valores = np.random.rand(Nind, Lind)
    muta = valores <= Pm
    nuevo_gen = np.logical_xor(nuevo_gen, muta)

    return nuevo_gen
```

Resultado esperado, implemente el código necesario para su graficación.

ráfica de f(x, y) = 21.5 + x * sin(4πx) + y * sin(20πy) con Algoritmos Genéti



Replique el código anterior, agregando las librerías que hagan falta.

Compare los resultados con la programación estructurada (sin AG) que a continuación se muestra:

```
import numpy as np
import matplotlib.pyplot as plt

# Definir la función
def f(x, y):
    return 21.5 + x * np.sin(4 * np.pi * x) + y * np.sin(20 * np.pi * y)

# Generar datos para x y y
x = np.linspace(-3, 12.1, 100)
y = np.linspace(4.1,5.8, 100)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)

# Crear la gráfica en 3D
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis')

# Etiquetas y título
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('f(x, y)')
ax.set_title('Gráfica de f(x, y) = 21.5 + x * sin(4πx) + y * sin(20πy)')

# Mostrar la gráfica
plt.show()
```